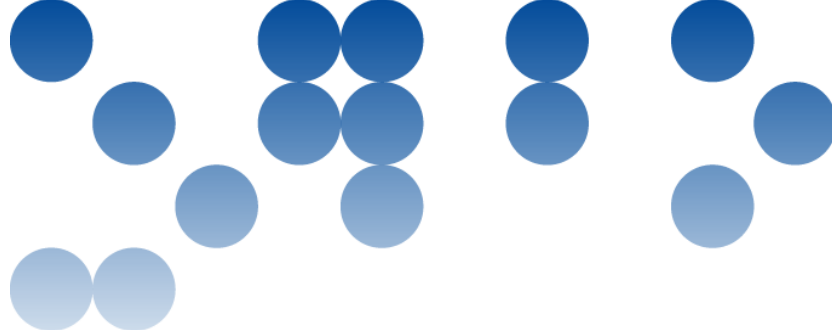




PEC2

NOMBRE: Luis Rodrigo
APELLIDOS: Martínez Tabernero

Estructura de computadores 2025-s2

Estudios de informática, multimedia y comunicación

Presentación

La presente PEC2 contiene 3 preguntas y representa el 50% de la nota de la evaluación continua.

Como podréis ver, los ejercicios son muy parecidos a los que habéis hecho durante estos días, en los que además habéis podido dar las soluciones, comentarlas y plantear dudas en el foro. Esta PEC es **individual**, **evaluable** y por tanto no puede comentarse.

Competencias

Las competencias específicas que persigue la PEC2 son:

- [13] Capacidad para identificar los elementos de la estructura y los principios de funcionamiento de un ordenador.
- [14] Capacidad para analizar la arquitectura y organización de los sistemas y aplicaciones informáticos en red.
- [15] Conocer las tecnologías de comunicaciones actuales y emergentes y saberlas aplicar convenientemente para diseñar y desarrollar soluciones basadas en sistemas y tecnologías de la información.

Objetivos

Los objetivos de la siguiente PEC son:

- Conocer la organización del sistema de memoria de un computador.
- Conocer el funcionamiento de la memoria cache, así como los algoritmos de correspondencia y reemplazamiento.
- Conocer la organización del sistema de entrada/salida.
- Comprender las técnicas de entrada/salida (entrada/salida programada, Interrupciones y DMA).

Enunciado

Responder cada pregunta o apartado en el recuadro correspondiente.

Recursos

Podéis consultar los recursos disponibles en el aula, pero no hacer uso del foro.

Criterios de valoración

La puntuación y los criterios de evaluación los encontraréis en cada pregunta.

Formato y fecha de entrega

La PEC2 se ha de entregar en el apartado de **entrega de actividades en un documento en formato pdf**. Es necesario que rellenéis en el espacio reservado de la primera página de la PEC vuestro **NOMBRE Y APELLIDOS**. La fecha **límite** de entrega es el **04/05/2026**.



Enunciado

Pregunta 1 (4 puntos)

Tenemos un sistema de memoria en el que todos los accesos se hacen a palabra (sin que importe el tamaño de una palabra). El espacio de direcciones de memoria se descompone en bloques de $B = 8$ palabras, lo que implica que cada bloque comienza en una dirección múltiplo de B . La fórmula para calcular el identificador numérico del bloque es:

Bloque = dirección de memoria (dirección palabra) DIV B (tamaño del bloque en palabras)

El sistema dispone de una memoria cache de $L = 4$ líneas, donde cada línea tiene el tamaño de un bloque. Estas líneas se identifican como líneas 0, 1, 2 y 3.

Apartado 1.1 (1,5 puntos) Memoria Cache de Acceso Directo

El sistema utiliza una **política de asignación directa**, de manera que cada bloque de la memoria principal sólo se puede llevar a una línea determinada de la memoria cache. El identificador del bloque determina la línea específica donde se puede guardar el bloque utilizando la siguiente fórmula:

Línea = identificador de bloque MOD L (tamaño de la cache en líneas)

La ejecución de un programa genera la siguiente lista de lecturas a memoria:

8, 9, 30, 2, 3, 12, 16, 1, 43, 44, 45, 14, 46, 18, 73, 15, 39, 60, 4, 35

La siguiente tabla muestra el estado inicial de la cache, que contiene las primeras $L \times B$ palabras de la memoria (organizadas en L bloques). Debéis completar la tabla para mostrar la evolución de la cache durante la ejecución del programa, indicando únicamente los fallos que se producen. Para cada fallo hay que llenar una columna indicando la dirección de la palabra accedida, el nuevo bloque que se trae a la memoria cache y a qué línea, expresado de la forma **b:e** ($a_1 - a_B$), donde **b**: número de bloque de memoria principal, **e**: etiqueta y ($a_1 - a_B$) es el rango de las direcciones del bloque, donde a_1 es la primera dirección del bloque y a_B es la última dirección del bloque.

VAMOS A PROBAR CADA NÚMERO DE LA LISTA PASO A PASO

-Dirección 8

Bloque $8//8 = 1$

Línea $1 \pmod{4} = 1$

La línea 1 ya tiene bloque 1. ACIERTO

-Dirección 9

Bloque $8//8 = 1$

Línea $1 \pmod{4} = 1$

La línea 1 ya tiene bloque 1. ACIERTO

-Dirección 30

Bloque 3

Línea 3

ACIERTO

-Dirección 2

Bloque 0

Línea 0

ACIERTO

-Dirección 3

Bloque 0

Línea 0

ACIERTO

-Dirección 12

Bloque 1

Línea 1

ACIERTO

-Dirección 16

Bloque 2

Línea 2

ACIERTO

-Dirección 1

Bloque 0

Línea 0

ACIERTO

-Dirección 43

Bloque $43//8 = 5$



Línea 5 (mod 4) = 1

Etiqueta = $5 // 4 = 1$

La línea 1 ya tiene bloque 1. FALLO

-Dirección 44

Bloque 5

Línea 1

ACIERTO

-Dirección 45

Bloque 5

Línea 1

ACIERTO

-Dirección 14

Bloque $14 // 8 = 1$

Línea 1 (mod 4) = 1

Etiqueta = $1 // 4 = 0$

La línea 1 tiene el bloque 5. FALLO

-Dirección 46

Bloque $46 // 8 = 5$

Línea 5 (mod 4) = 1

Etiqueta = $5 // 4 = 1$

La línea 1 tiene el bloque 1. FALLO

-Dirección 18

Bloque 2

Línea 2

ACIERTO

-Dirección 73

Bloque $73//8 = 9$

Línea $9 \pmod{4} = 1$

Etiqueta = $9//4 = 2$

La línea 1 tiene el bloque 5. FALLO

-Dirección 15

Bloque $15//8 = 1$

Línea $1 \pmod{4} = 1$

Etiqueta = $1//4 = 0$

La línea 1 tiene el bloque 9. FALLO

-Dirección 39

Bloque $39//8 = 4$

Línea $4 \pmod{4} = 0$

Etiqueta = $4//4 = 1$

La línea 0 tiene el bloque 0. FALLO

-Dirección 60

Bloque $60//8 = 7$

Línea $7 \pmod{4} = 3$

Etiqueta = $7//4 = 1$

La línea 3 tiene el bloque 3. FALLO

-Dirección 4

Bloque $4//8 = 0$

Línea $0 \pmod{4} = 0$

Etiqueta = $0//4 = 0$

La línea 0 tiene el bloque 4. FALLO

-Dirección 35

Bloque $35//8 = 4$

Línea $4 \pmod{4} = 0$



Etiqueta = $4/4 = 1$

La línea 0 tiene el bloque 0. FALLO

Línea	Estado Inicial	Fallo: 43	Fallo: 14	Fallo: 46	Fallo: 73	Fallo: 15
0	0:0 (0 - 7)					
1	1:0 (8 - 15)	5:1 (40-47)	1:0 (8 - 15)	5:1 (40-47)	9:2 (72-79)	1:0 (8 - 15)
2	2:0 (16 - 23)					
3	3:0 (24 - 31)					

Línea	Fallo: 39	Fallo: 60	Fallo: 4	Fallo: 35	Fallo:	Fallo:
0	4:1 (32-39)		0:0 (0-7)	4:1 (32-39)		
1						
2						
3		7:1 (56-63)				

Apartado 1.2 (1,5 puntos) Memoria cache de acceso completamente asociativo

Ahora suponemos que en el mismo sistema se utiliza una política de emplazamiento completamente asociativa, de manera que cualquier bloque de la memoria principal se puede llevar a cualquier línea de la memoria cache. Si encontramos que la cache ya está llena, se utiliza un **algoritmo de reemplazamiento LRU**, de manera que sacaremos de la memoria cache aquel bloque que hace más tiempo que no se referencia.

Consideremos la misma lista de lecturas a memoria:

8, 9, 30, 2, 3, 12, 16, 1, 43, 44, 45, 14, 46, 18, 73, 15, 39, 60, 4, 35

De nuevo, debéis completar la tabla para mostrar la evolución de la cache durante la ejecución del programa, indicando únicamente los fallos que se producen. Suponemos que el orden en que se ha accedido por última vez a los bloques 0, 1, 2 y 3, que están en caché, es 0, 1, 2 y 3: es decir, el bloque 3 es el que se ha accedido más recientemente y el bloque 0 es el que se ha accedido hace más tiempo. Para cada fallo hay que llenar una columna indicando la dirección de la palabra accedida, el nuevo bloque que se trae a la memoria cache y a qué línea, expresado de la forma **b** ($a_1 - a_B$), donde **b**: número de bloque, y ($a_1 - a_B$) son las direcciones del bloque, donde a_1 es la primera dirección del bloque y a_B es la última dirección del bloque.

-Dirección 8

Bloque 1. Acierto.

Prioridad actualizada: 0,2,3,1

-Dirección 9

Bloque 1. Acierto.

Prioridad actualizada: 0,2,3,1

-Dirección 30

Bloque 3. Acierto.

Prioridad actualizada: 0,2,1,3

-Dirección 2

Bloque 0. Acierto.

Prioridad actualizada: 2,1,3,0

-Dirección 3

Bloque 0. Acierto.

Prioridad actualizada: 2,1,3,0

-Dirección 12

Bloque 1. Acierto.

Prioridad actualizada: 2,3,0,1

-Dirección 16

Bloque 2. Acierto.

Prioridad actualizada: 3,0,1,2

-Dirección 1

Bloque 0. Acierto.

Prioridad actualizada: 3,1,2,0

-Dirección 43

Bloque 5. FALLO.

Expulsamos al 3, que encabeza la prioridad.



Prioridad actualizada: 1,2,0,5

-Dirección 44

Bloque 5. Acierto.

Prioridad actualizada: 1,2,0,5

-Dirección 45

Bloque 5. Acierto.

Prioridad actualizada: 1,2,0,5

-Dirección 14

Bloque 1. Acierto.

Prioridad actualizada: 2,0,5,1

-Dirección 46

Bloque 5. Acierto.

Prioridad actualizada: 2,0,1,5

-Dirección 18

Bloque 2. Acierto.

Prioridad actualizada: 0,1,5,2

-Dirección 73

Bloque 9. FALLO.

Prioridad actualizada: 1,5,2,9

-Dirección 15

Bloque 1. Acierto.

Prioridad actualizada: 5,2,9,1

-Dirección 39

Bloque 4. FALLO

Prioridad actualizada: 2,9,1,4

-Dirección 60

Bloque 7. FALLO.

Prioridad actualizada: 9,1,4,7

-Dirección 4

Bloque 0. FALLO.

Prioridad actualizada: 1,4,7,0

-Dirección 35

Bloque 4. Acierto.

Prioridad actualizada: 1,7,0,4

Línea	Estado Inicial	Fallo: 43	Fallo: 73	Fallo: 39	Fallo: 60	Fallo: 4	Fallo:
0	0 (0 - 7)	0(0-7)	9 (72-79)	9 (72-79)	9 (72-79)	0 (0-7)	
1	1 (8 - 15)	1 (8-15)	1 (8-15)	1 (8-15)	1 (8-15)	1 (8-15)	
2	2 (16 - 23)	2 (16-23)	2 (16-23)	2 (16-23)	7 (56-63)	7 (56-63)	
3	3 (24 - 31)	5 (40-47)	5 (40-47)	4 (32-39)	4 (32-39)	4 (32-39)	

Línea	Fallo:	Fallo:	Fallo:	Fallo:	Fallo:	Fallo:
0						
1						
2						
3						

Apartado 1.3 (1 punto) Rendimiento de la Memoria cache

Suponemos que en un total de 60 accesos se producen 22 fallos en caché. Además, el tiempo total de acceso en caso de fallo (t_f) es 35 ns, y el tiempo de acceso a la memoria cache, también denominado tiempo de acceso en caso de acierto (t_a), es 3 ns.

1.3.a) (0,4 puntos)

Calcula la tasa de fallos (T_f).

$$T_f = \text{Fallos/Accesos} = 22/60 = 0.3667 = 36,67\%$$

1.3.b) (0,6 puntos)

Considerando la tasa de fallos obtenida en la pregunta anterior, calcula el tiempo medio de acceso a memoria (t_m).

$$T_m = (T_a \cdot t_a) + (T_f \cdot t_f) = (1 - 0,3667) \cdot 3 \text{ ns} + (0,3667 \cdot 35 \text{ ns}) = 1,9 + 12,83 = 14,73 \text{ ns}$$



Criterios de valoración. Para los apartados 1.1 y 1.2 cada error en los fallos o aciertos de la memoria cache o en la colocación de un bloque en la cache resta 0,5. Los dos apartados finales están bien o están mal: no hay gradación.

Pregunta 2 (5 puntos)

Se desea analizar el rendimiento de la comunicación de datos entre un dispositivo conectado a un puerto USB y la memoria de un procesador. El sistema tiene las siguientes características:

- Velocidad de transferencia del dispositivo de E/S (v_{transf})= 1 MBytes/s ($M=10^6$)
- Tiempo de latencia promedio del dispositivo (t_{latencia}) = 0
- Direcciones de los **registros de estado y de datos** del controlador de E/S: AF04h y AF00h, respectivamente.
- El bit del **registro de estado** que indica que el controlador del puerto de E/S está disponible es el bit 3 (cuarto bit menos significativo) que cuando vale 1 indica que el puerto está disponible
- La frecuencia de reloj del procesador es 1 GHz, y el procesador puede ejecutar 1 instrucción cada 4 ciclos de reloj ($t_{\text{instr}} = 4 \times t_{\text{ciclo}}$)
- Transferencia de **escritura** desde la memoria al puerto de E/S
- Cada dato transferido son 4 Bytes y la transferencia es de $N_{\text{datos}} = 1.600.000$ datos
- Dirección inicial de memoria donde residen los datos: 10000000h

Apartado 2.1 (2 puntos) E/S programada

2.1.a) (0,5 puntos)

El siguiente código realizado con el repertorio CISCA realiza la transferencia descrita antes mediante la técnica de **E/S programada**. Debéis completar el siguiente código:

```
1.      MOV    R3, 1600000
2.      MOV    R2, 10000000h
3. Bucle: IN     R0, AF04h
4.      AND    R0, 8
5.      JE     Bucle
6.      MOV    R0, [R2]
7.      ADD    R2, 4
8.      OUT    [AF00h], R0
9.      SUB    R3, 1
10.     JNE    Bucle
```

2.1.b) (0,5 puntos)

¿Cuánto tiempo dura la transferencia $t_{\text{transf_bloque}}$?

¿Qué porcentaje de este tiempo dedica la CPU a la transferencia?

$V_{\text{transf}} = 1\text{MB/s} = 10^6 \text{ bytes/s}$

$\text{Tamaño_Dato} = 4 \text{ bytes.}$

$\text{Tiempo_dato} = 4/10^6 = 4 \text{ microsegundos.}$

$T_{\text{transf_bloque}} = N_{\text{datos}} * t_{\text{dato}} = 1600000 * 4 \text{ microseg.} = 6,4 \text{ segundos.}$

Ocupación CPU: 100%, en programada siempre hay espera activa.

2.1.c) (1 punto)

Si quisiéramos utilizar el mismo procesador y programa, pero con un dispositivo de E/S más rápido, ¿Cuál es la máxima tasa o velocidad de transferencia del nuevo dispositivo que se podría soportar sin que el dispositivo se tuviera que esperar?



La frecuencia del procesador es $1\text{GHz} = 10^9\text{ Hz}$

Tiempo_ciclo = $1/\text{frecuencia} = 1\text{ ns}$

El procesador ejecuta 1 instrucción cada 4 ciclos de reloj:

Tiempo_instr = $4 * \text{tiempo_ciclo} = 4\text{ns}$

Tiempo mínimo del bucle de programa:

Vamos de la instrucción 3 a la 10 en el bucle. Asumimos que las instrucciones se ejecutan una vez por dato.

Tiempo_min_dato = $8 \text{ instrucciones} * 4\text{ns/instr} = 32\text{ ns}$

Velocidad máxima soportada:

Se pueden transferir 4 bytes cada 32 ns

$V_{\text{max}} = 4 \text{ bytes} / (32 * 10^{-9} \text{ segs}) = 125000000 \text{ bytes/s} = 125 \text{ Mbytes/s}$

Apartado 2.2 (2 puntos) E/S por Interrupciones

Suponer que el siguiente código CISCA es una rutina de servicio a las interrupciones (RSI) para transferir a través del dispositivo de E/S anterior, el mismo número de datos que antes (E/S programada), pero ahora mediante la técnica de E/S por interrupciones.

Suponer:

- Tiempo de programación y finalización de la transferencia de 1 microsegundo ($t_{\text{prog}} + t_{\text{final}}$)
- El tiempo para atender la interrupción ($t_{\text{rec_int}}$), o tiempo adicional desde que la CPU detecta la interrupción hasta que comienza a ejecutarse la primera instrucción de la RSI es 20 ciclos de reloj.
- Se utiliza una variable global que se representa con la etiqueta **DIR**, y que al principio del programa contiene la dirección inicial de memoria donde residen los datos a transferir.

2.2.a) (0,5 puntos)

Completar el código:

```
1.  CLI
2.  PUSH      R0
3.  PUSH      R1
4.  IN        R0, 8
5.  AND       R0, 8
6.  JE        Error ; salta a un código no descrito de tratamiento de error:
                        ; el dato no está disponible.
7.  MOV       R1, [DIR]
8.  MOV       R0, [R1]
9.  OUT       AF00h, R0
10. ADD      R1, 4
11. MOV      [DIR] , R1
12. POP      R1
13. POP      R0
14. STI
15. IRET
```

2.2.b) (1 punto)

¿Cuál es el tiempo total que dedica la CPU a la tarea de Entrada/Salida, t_{cpu} ? ¿Qué porcentaje representa el tiempo de transferencia del bloque $t_{\text{transf_bloque}}$ respecto al tiempo de transferencia del bloque por parte del periférico t_{bloque} ?

$T_{\text{ciclo}} = 1 \text{ ns} \rightarrow \text{Lo sacamos de } \text{frec.} = 1 \text{ GHz}$

$T_{\text{instr}} = 4 * t_{\text{ciclo}} = 4 \text{ ns}$

$T_{\text{rec_int}} = 20 \text{ ciclos} * 1 \text{ ns} / \text{ciclo} = 20 \text{ ns}$

El código tiene 15 instrucciones

$T_{\text{rsi}} = 15 \text{ instr} * 4 \text{ ns/instr} = 60 \text{ ns}$

$T_{\text{total_trasf_dato}} = 20 \text{ ns} + 60 \text{ ns} = 80 \text{ ns}$

La finalización tarda 1 microSegundo = 1000ns

Transferencia total: $1.600.000 \text{ datos} * 80 \text{ ns/dato} = 128000000 \text{ ns} = 0,128 \text{ s}$

$T_{\text{cpu}} = 1000 \text{ ns} + 128000000 \text{ ns} = \text{Más o menos lo mismo, } 0,128 \text{ s}$

**2.2.c) (0,5 puntos)**

Si quisiéramos reducir la frecuencia de reloj del procesador para reducir su consumo energético, ¿hasta qué frecuencia lo podríamos hacer sin reducir la velocidad de transferencia con el dispositivo de E/S?

Hemos de ser capaz de llevar una interrupción antes de que llegue la siguiente.

Frecuencia del periférico: $T_{\text{dato}} = 4 \text{ microseg.} = 4000 \text{ ns}$

Tiempo de ciclo: $t_{\text{rec_int}} = 20 * t_c$

$T_{\text{rsi}} = 15 \text{ instr} * (4 * t_c) = 60 * t_c$

$T_{\text{total}} = 80 * t_c$

$80 * t_c = 4000 \text{ ns} \rightarrow t_c = 4000/80 = 50\text{ns}$

Frec_min: $f = 1 / (50 * 10^{-9}) = 20000000 \text{ Hz} = 20 \text{ MHz}$

Apartado 2.3 (1 punto) E/S por DMA

Supondremos que el controlador de E/S puede funcionar en modo DMA (Acceso Directo a Memoria). La suma del **tiempo de cesión** del bus y del **tiempo de recuperación** del bus es 20 ns ($t_{\text{cesión}} + t_{\text{recup}}$). El **tiempo de la transferencia** de un dato por el bus es 1 ns (t_{mem}).

2.3.a) (0,5 puntos)

Consideremos que en la transferencia por DMA, los datos se envían entre el controlador de DMA y la memoria, en modo ráfaga, y dispone de un buffer de un tamaño $m_{\text{buffer}} = 2.000$ Bytes. Calcular el tiempo total de ocupación del bus por parte del controlador de DMA para llevar a cabo la transferencia que venimos analizando.

Tamaño del dato: 4 bytes

Nráfaga = $m_{\text{buffer}} / m_{\text{dato}} = 2000/4 = 500$ datos por ráfaga

Tiempo por ráfaga:

$T_{\text{cesión}} + t_{\text{recuperación}} = 20$ ns

T_{mem} : 1ns por dato

$T_{\text{trans_ráfaga}} = (t_{\text{cesión}} + t_{\text{rec}}) + (N_{\text{ráfaga}} * t_{\text{mem}}) = 20 \text{ ns} + (500 * 1 \text{ ns}) = 520 \text{ ns}$

Tiempo total por bloque:

Ntotal de ráfagas: $N_{\text{datos}}/N_{\text{ráfaga}} = 1.600.000/500 = 3200$ ráfagas

$T_{\text{transf_bloque}} = 3200 \text{ ráfagas} * 520 \text{ ns/ráfaga} = 1.664.000 \text{ ns} = 1,664 \text{ ms}$

2.3.b) (0,5 puntos)

La CPU no puede hacer ninguna tarea durante todo el tiempo en que el bus está ocupado por parte del controlador de DMA. ¿Cuál es el porcentaje de tiempo que tiene disponible la CPU para ejecutar código efectivo de otros programas durante la transferencia?

Mientras que en la programada estaba libre el 0% del tiempo, en este caso de DMA solo detenemos el procesador cuando el bus está ocupado por DMA. El resto es libre.

En el apartado 2.1.b ya está calculado que el periférico tarda 6,4 segs. En completar la tarea.

$\% \text{ocupación} = t_{\text{trasf_bloque}} / t_{\text{bloque}} * 100 = (1,664 \text{ ms} * 100) / 6400 = 0,026\%$

Luego el disponible es $100\% - \text{ocupación} = 99,974\%$

Esto es una mejora notoria frente a Programada.

Criterios de valoración. En los apartados 2.1.a y 2.2.a cada valor erróneo resta 0,25. El resto de los apartados están bien o están mal: no hay gradación.



Pregunta 3 (1 punto)

Preguntas teóricas

3.a) (0,25 puntos)

Según el documento sobre el módulo 4 hay dos formas de ordenar en la memoria los bytes de una palabra que ocupa múltiples bytes. Indica cuáles son estas dos formas y pon un ejemplo con la palabra 1234h, almacenada en la dirección 100h

Las dos formas son Little-Endian, donde el byte de menor peso se coloca en la dirección menor, y Big-Endian, donde el byte de mayor peso se coloca en la dirección menor.

- Para el ejemplo 1234h en 100h: Little-Endian sería (100h: 34h, 101h: 12h); Big-Endian (100h: 12h, 101h: 34h)

3.b) (0,25 puntos)

Según el documento sobre el módulo 4, ¿cuáles son los posibles algoritmos de reemplazo para una memoria caché de asignación directa?

- Ninguno, es una pregunta trampa, porque como su nombre indica, la asignación de bloques y líneas ya está asignada. El reemplazo es forzoso y automático

3.c) (0,25 puntos)

En un sistema de E/S gestionado por DMA ¿Qué diferencia hay entre el funcionamiento normal y el modo ráfaga? ¿qué ventajas tiene un modo con respecto al otro?

En normal se pide el bus por cada dato, mientras que en ráfaga va por bloques. El modo ráfaga tiene la ventaja de que es más eficiente porque solo necesita pedir el bus al procesador cuando su buffer interno se llena, eliminando esperas de cesión y recuperación. Como no pueden usar el bus a la vez, al liberar antes el bus el DMA, nuestro procesador puede volver a ejecutar programas con normalidad.

3.d) (0,25 puntos)

En la E/S programada, ¿Cómo se realiza la transferencia de un dato? Describir el proceso.

PASO 1: La CPU lee el registro de estado del periférico

PASO 2: Se comprueba el bit de "listo"

PASO 3: Si no está listo, se vuelve al paso 1 a esperar activamente.

PASO 4: En el caso de que esté listo, transfiere el dato entre los registros de datos del periférico y la CPU.